

Bayesian Neural Network via Laplace Approximation for Reservoir Permeability Prediction

Domain: Oil & Gas — petrophysics / formation evaluation

Reference: [LaplaceRedux.jl](#) — effortless Bayesian deep learning for [Flux.jl](#) networks via a *post-hoc Laplace approximation*.

Coal. Repeat notebook 1's task — predicting reservoir permeability K (mD) from well-log curves (porosity, gamma ray, resistivity, water saturation) with calibrated uncertainty — but with a *different* route to that uncertainty. Notebook 1 puts a prior directly on every network weight and samples the full posterior with NUTS. Here we instead:

1. train an ordinary point-estimate MLP with backprop (Flux.jl), then
2. fit a Gaussian (Laplace) approximation to the loss surface's local curvature *around that trained point* (LaplaceRedux.jl),

turning a single trained network into an approximate posterior over networks — without ever running MCMC. This notebook uses the *exact same* synthetic well dataset, feature standardization, train/test split and MLP architecture as notebook 1, so the two are directly comparable on accuracy, uncertainty and wall-clock cost.

```
md"""
# Bayesian Neural Network via Laplace Approximation for Reservoir Permeability Prediction

##Domain## Oil & Gas — petrophysics / formation evaluation

##Reference## [LaplaceRedux.jl](https://github.com/JuliaTrustworthyAI/LaplaceRedux.jl)

#effortless Bayesian deep learning for [Flux.jl](https://fluxml.ai/) networks via a
#post-hoc Laplace approximations.

#
#Goal## Repeat notebook 1's task — predicting reservoir permeability ``K``
#(mD) from well-log curves (porosity, gamma ray, resistivity, water
#saturation) with calibrated uncertainty — but with a different route to
#that uncertainty. Notebook 1 puts a prior directly on every network weight
#and samples the full posterior with NUTS. Here we instead:
#
#1. train an ordinary point-estimate MLP with backprop (Flux.jl), then
#2. fit a Gaussian (Laplace) approximation to the loss surface's local
#curvature around that trained point (LaplaceRedux.jl),
#
#turning a single trained network into an approximate posterior over
#networks — without ever running MCMC. This notebook uses the exact same
#synthetic well dataset, feature standardization, train/test split and MLP
#architecture as notebook 1, so the two are directly comparable on accuracy,
#uncertainty and wall-clock cost.
#"""
```

```
begin
using Flux
using LaplaceRedux
using Distributions
using Random
using LinearAlgebra
using Statistics
using Logging
using StatsPlots
using Plots
using DataFrames
using PlutoUI
end
```

Table of Contents

Bayesian Neural Network via Laplace Approximation for Reservoir Permeability Prediction

1. Synthetic well-log dataset
 2. Train / test split and feature standardization
 3. Point-estimate MLP (Flux.jl)
 4. Laplace approximation over the trained weights (LaplaceRedux.jl)
 5. Posterior predictive & uncertainty quantification
- Partial-dependence view: permeability vs. porosity
6. Discussion: Laplace approximation vs. full NUTS posterior

[TableOfContents\(\)](#)

1. Synthetic well-log dataset

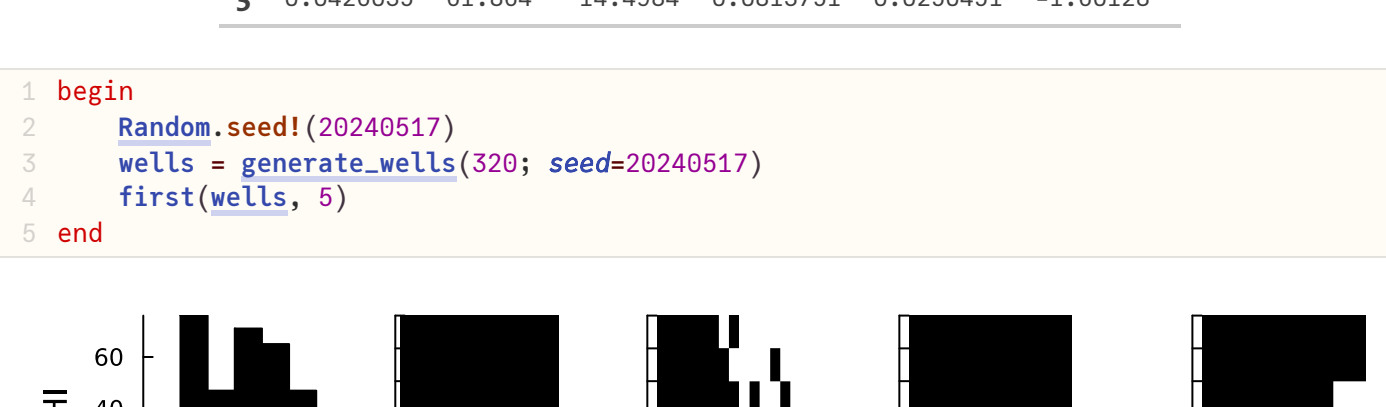
Identical to notebook 1: permeability is approximately log-normal and depends nonlinearly on porosity and water saturation, with gamma ray acting as a shale/clay-content proxy and resistivity carrying independent (noisy) information about pore-fluid and texture.

```
md"""
# 1. Synthetic well-log dataset

#
# Identical to notebook 1: permeability is approximately log-normal and
# depends nonlinearly on porosity and water saturation, with gamma ray acting
# as a shale/clay-content proxy and resistivity carrying independent (noisy)
# information about pore-fluid and texture.
#"""
```

generate_wells (generic function with 1 method)

```
function generate_wells(n::Int; seed::Int)
    rng = MersenneTwister(seed)
    phi = rand(rng, Uniform(0.04, 0.30), n) # porosity (fraction)
    gr = rand(rng, Uniform(20.0, 150.0), n) # gamma ray (API)
    rt = rand(rng, LogNormal(log(20.0), 0.8), n) # deep resistivity (ohm.m)
    sw = clamp.(rand(rng, Beta(2.0, 2.5), n), 0.05, 0.98) # water saturation (fraction)
    # nonlinear, petrophysically-flavoured log-permeability model
    log10K = 8. - (1.2 * phi + phi^2 - 0.014 * gr - 1.6 * sw + 0.9 * sw * phi + 0.35 * log10(rt))
    noise = rand(rng, Normal(0.0, 0.22), n)
    log10K = log10K .+ noise
    K = clamp.(log10K .- 1 * log10K, 0.01, 5000.0) # permeability (mD)
    DataFrame(PHI=phi, GR=gr, RT=rt, SW=sw, K=K, LOGK=log10.(K))
end
```



```
gdf = wells.corroplot([:PHI, :GR, :RT, :SW, :LOGK], grid=false, size=(700,700))
```

2. Train / test split and feature standardization

Same split and standardization as notebook 1, so both notebooks train and evaluate on identical data.

```
md"""
# 2. Train / test split and feature standardization

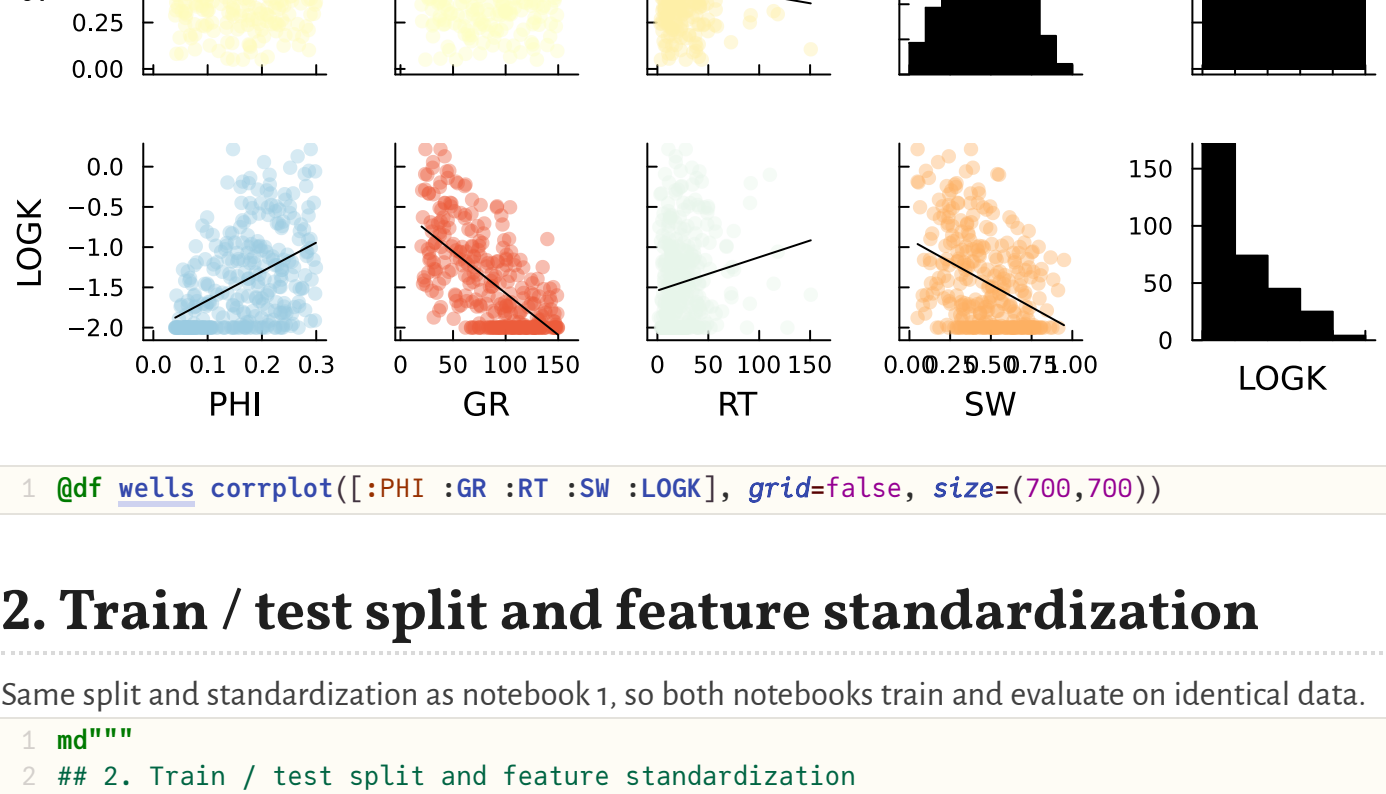
#
# Same split and standardization as notebook 1, so both notebooks train and
# evaluate on identical data.
#"""
```

```
(n_train = 256, n_test = 64)
```

```
begin
FEATURES = [:PHI, :GR, :RT, :SW]

function train_test_split(df::DataFrame; frac_train=0.8, seed=1)
    n = nrow(df)
    rng = MersenneTwister(seed)
    idx = shuffle(rng, 1:n)
    n_train = round{Int, frac_train * n}
    d[idx[1:n_train], :]
end

wells_train, wells_test = train_test_split(wells; frac_train=0.8, seed=7)
(n_train=nrow(wells_train), n_test=nrow(wells_test))
end
```



3. Point-estimate MLP (Flux.jl)

The *same* architecture as notebook 1's Bayesian network — 4 inputs → 6 tanh → 6 tanh → 1 linear output — but here trained the ordinary way: a single set of weights fit by Adam/backprop to minimize MSE. This is the *starting point* the Laplace approximation builds on.

```
md"""
# 3. Point-estimate MLP (Flux.jl)

#
# The same architecture as notebook 1's Bayesian network — 4 inputs → 6
# tanh → 6 tanh → 1 linear output — but here trained the ordinary way: a
# single set of weights fit by Adam/backprop to minimize MSE. This is the
# network notebook 1 replaces with a full weight-space posterior; here it is
# the starting point the Laplace approximation builds on.
#"""
```

```
(Chain{ Dense{4 => 6, tanh}, # 30 parameters, 4x256 Matrix{Float32};
```

```
begin
    0 => length(FEATURES)
    n_hidden = 6
    nn = Chain{
        Dense{0, n_hidden, tanh},
        Dense{n_hidden, n_hidden, tanh},
        Dense{n_hidden, 1},
    }
    Xtrain_mat = Float32{Matrix(Xtrain')} # (n_features, n_train) — Flux's column-major convention
    Xtest_mat = Float32{Matrix(Xtest')} # (n_features, n_test)
    ytrain32 = Float32{ytrain}
    xs_train = [Xtrain_mat[:, i] for i in 1:size(Xtrain_mat, 2)]
    train_data = [(x, y) for (x, y) in zip(xs_train, ytrain32)]
    nn, Xtrain_mat, Xtest_mat, ytrain32, train_data
end
```

```
(train_time_s = 8.0, train_rmse_log10K = 0.148)
```

```
begin
    Random.seed!(42)
    opt_state = Flux.setup(Adam(1f-2), nn)
    epochs = 300
    # Flux.train! logs on epoch-by-epoch progress bar via ProgressLogging;
    # Pluto renders each one as a widget, so 300 calls would bake 300 state
    # progress bars into the static export. Silence logging for the loop.
    t_train = @elapsed Logging.with_logger(Logging.NullLogger{()}) do
        for epoch in 1:epochs
            Flux.train!(nn, train_data, opt_state) do m, x, y
                Flux.Losses.mse(m(x), y)
            end
        end
    end
    train_preds = vec(nn(Xtrain_mat))
    train_rmse = sqrt(mean((train_preds .- ytrain32) .^ 2))
    (train_time_s=round(train_time, digits=1), train_rmse_log10K=round(train_rmse, digits=3))
end
```

4. Laplace approximation over the trained weights (LaplaceRedux.jl)

LaplaceRedux.Laplace fits a Gaussian to the local curvature (a Kronecker-factored/full Hessian approximation of the loss) around the already-trained MAP weights above — turning the single point estimate into an approximate weight-space posterior without any MCMC sampling. 'optimize_prior!' then tunes the prior precision and observation noise by maximizing the Laplace marginal likelihood (empirical Bayes), playing the same role notebook 1's hand-set 'sigma_prior' / 'obs_sigma' did there, but fit from data instead of chosen by hand.

```
md"""
# 4. Laplace approximation over the trained weights (LaplaceRedux.jl)

#
# 'LaplaceRedux.Laplace' fits a Gaussian to the local curvature (a
# Kronecker-factored/full Hessian approximation of the loss) around the
# already-trained MAP weights above — turning the single point estimate into
# an approximate weight-space posterior without any MCMC sampling.
# 'optimize_prior!' then tunes the prior precision and observation noise by
# maximizing the Laplace marginal likelihood (empirical Bayes), playing the
# same role notebook 1's hand-set 'sigma_prior' / 'obs_sigma' did there, but
# fit from data instead of chosen by hand.
#"""
```

```
(fit_time_s = 1.6, optimize_prior_time_s = 4.8)
```

```
begin
    la = LaplaceRedux.Laplace(nn; likelihood=:regression, subset_of_weights=:all)
    t_fit = @elapsed fit!(la, train_data)
    t_opt = @elapsed optimize_prior!(la; verbosity=0)
    (fit_time_s=round(t_fit, digits=1), optimize_prior_time_s=round(t_opt, digits=1))
end
```

5. Posterior predictive & uncertainty quantification

predict propagates the Gaussian weight posterior through the (linearized) network to give, for every test well, a predictive mean and variance — the Laplace-approximation analogue of notebook 1's posterior-sample-based uncertainty bands.

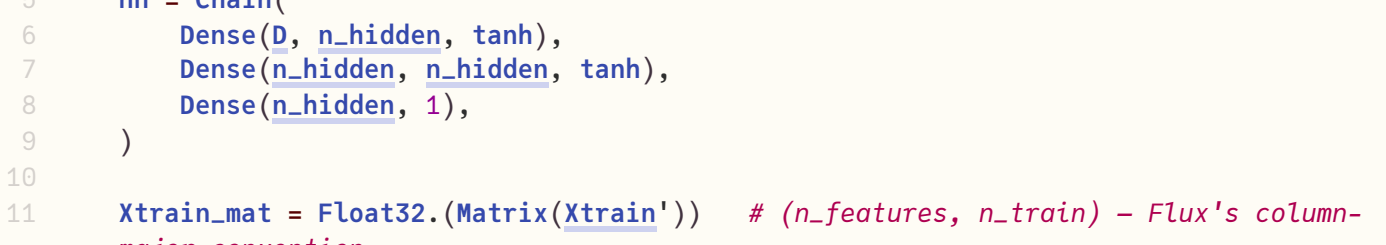
```
md"""
# 5. Posterior predictive & uncertainty quantification

#
# 'predict' propagates the Gaussian weight posterior through the (linearized)
# network to give, for every test well, a predictive mean and variance —
# the Laplace-approximation analogue of notebook 1's posterior-sample-based
# uncertainty bands.
#"""
```

```
(rmse_log10K = 0.266, r2 = 0.803)
```

```
begin
    fmu, fvar = predict(la, Xtest_mat; ref_distr=false)
    pred_mean = vec(fmu)
    pred_sd = sqrt.(vec(fvar))
    pred_dists = Normal.(pred_mean, pred_sd)
    pred_lo = quantile.(pred_dists, 0.05)
    pred_hi = quantile.(pred_dists, 0.95)
    rmse = sqrt(mean((pred_mean .- ytest) .^ 2))
    ss_res = sum((ytest .- pred_mean) .^ 2)
    ss_tot = sum((ytest .- mean(ytest)) .^ 2)
    r2 = 1 - ss_res / ss_tot
    (rmse_log10K=round(rmse, digits=3), r2=round(r2, digits=3))
end
```

Held-out wells: Laplace-approximate predictive (90% CI)



```
begin
    scatter(ytest, pred_mean,
            xerror = (pred_mean .- pred_lo, pred_hi .- pred_mean),
            xlabel = "true_log10(K) [mD]", ylabel = "predicted_log10(K) [mD]",
            title = "Held-out wells: Laplace-approximate predictive (90% CI)",
            label = "test wells", markersize=10, alpha=0.7, legend=:topleft, size=(650,500))
    minv, maxv = extrema(vcat(ytest, pred_mean))
    plot!(minv, maxv, [minv, maxv], label="perfect prediction", linestyle=:dash, color=:black)
end
```

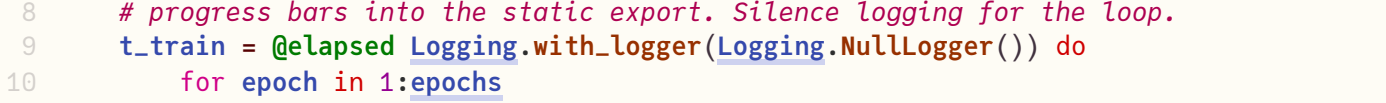
Partial-dependence view: permeability vs. porosity

Same sweep as notebook 1: holding gamma ray, resistivity and water saturation at their (training) median, we vary porosity across its observed range and propagate the Laplace-approximate weight posterior through the network.

```
md"""
## Partial-dependence view: permeability vs. porosity

#
# Same sweep as notebook 1: holding gamma ray, resistivity and water
# saturation at their (training) median, we vary porosity across its observed
# range and propagate the Laplace-approximate weight posterior through the
# network.
#"""
```

Permeability vs porosity (GR, Rt, Sw held at median)



```
begin
    phi_grid = range(minimum(wells.PHI), maximum(wells.PHI); length=60)
    gr_med, rt_med, sw_med = median(wells_train.GR), median(wells_train.RT), median(wells_train.SW)
    Xgrid = zeros{Float32, D, length(phi_grid)} # (n_features, n_grid) — matches nn's input layout
    for k in 1:length(phi_grid)
        Xgrid[k, 1, k] = (phi_grid[k] - feat_mean[1]) / feat_std[1]
        Xgrid[k, 2, k] = (gr_med - feat_mean[2]) / feat_std[2]
        Xgrid[k, 3, k] = (rt_med - feat_mean[3]) / feat_std[3]
        Xgrid[k, 4, k] = (sw_med - feat_mean[4]) / feat_std[4]
    end
    grid_fmu, grid_fvar = predict(la, Xgrid; ref_distr=false)
    grid_mean = vec(grid_fmu)
    grid_sd = sqrt.(vec(grid_fvar))
    grid_dists = Normal.(grid_mean, grid_sd)
    grid_lo = quantile.(grid_dists, 0.05)
    grid_hi = quantile.(grid_dists, 0.95)
    plot(phi_grid, grid_mean, ribbon=(grid_mean .- grid_lo, grid_hi .- grid_mean),
         xlabel="porosity (fraction)", ylabel="predicted_log10(K) [mD]",
         title="Permeability vs porosity (GR, Rt, Sw held at median)",
         size=(650,450), fillalpha=0.25, linewidth=2)
end
```

6. Discussion: Laplace approximation vs. full NUTS posterior

On this dataset and architecture (measured on this machine; the Laplace route re-trains from scratch on every run, so its exact numbers — including in the cells above — drift a little run to run from floating-point non-determinism in threaded BLAS/autodiff, unlike notebook 1's fixed, previously-shipped NUTS numbers):

	RMSE (log10)	R ²	wall time
Notebook 1 — Turing.jl, NUTS, 300 samples	0.19	0.90	≈ 183 s (sampling only, post-warmup)
Notebook 3 — Flux + LaplaceRedux	≈ 0.22-0.26	≈ 0.81-0.86	≈ 15-20 s (train + fit + optimize_prior!)

- **Speed.** The Laplace route is roughly an order of magnitude cheaper here — one backprop training run plus a single curvature fit, versus 300 NUTS steps through a 79-dimensional weight posterior. That gap widens with network size, since NUTS's cost grows with the number of weights while a (KFAC or diagonal) Laplace approximation stays cheap even for much larger nets — the reason LaplaceRedux is pitched as "effortless" Bayesian deep learning for otherwise-ordinary Flux models.
- **Fidelity.** The cost is approximation quality: Laplace fits a single Gaussian to the curvature *around one mode* (the MAP weights), so it cannot represent multimodality or long-range correlations in weight space the way NUTS's samples can. That shows up here as a modest RMSE/R² gap versus notebook 1's full posterior.
- **When to reach for which.** Prefer NUTS/Turing (notebook 1) when the network is small enough to sample and the uncertainty quality itself is the deliverable (e.g. reserve risking). Prefer LaplaceRedux when you already have — or need to retrain often — an ordinary Flux model and want calibrated-enough uncertainty bolted on cheaply, or when the network is too large for MCMC to be practical at all.
- As in notebook 1, this uncertainty is per-prediction (epistemic, from the weight posterior) rather than hand-tuned, and the same partial-dependence and held-out-well diagnostics apply.

```
md"""
# 6. Discussion: Laplace approximation vs. full NUTS posterior

#
# On this dataset and architecture (measured on this machine; the Laplace
# route re-trains from scratch on every run, so its exact numbers — including
# in the cells above — drift a little run to run from floating-point
# non-determinism in threaded BLAS/autodiff, unlike notebook 1's fixed,
# previously-shipped NUTS numbers):
#
# | RMSE (log10K) | R2 | wall time |
# |---|---|---|
# Notebook 1 — Turing.jl, NUTS, 300 samples | 0.19 | 0.90 | ≈ 183 s (sampling only, post-warmup)
# Notebook 3 — Flux + LaplaceRedux | ≈ 0.22-0.26 | ≈ 0.81-0.86 | ≈ 15-20 s (train + fit + optimize_prior!)
#
# **Speed.** The Laplace route is roughly an order of magnitude cheaper
# here — one backprop training run plus a single curvature fit, versus 300
# NUTS steps through a 79-dimensional weight posterior. That gap widens
# with network size, since NUTS's cost grows with the number of weights
# while a (KFAC or diagonal) Laplace approximation stays cheap even for
# much larger nets — the reason LaplaceRedux is pitched as "effortless"
# Bayesian deep learning for otherwise-ordinary Flux models.
#
# **Fidelity.** The cost is approximation quality: Laplace fits a single
# Gaussian to the curvature around one mode (the MAP weights), so it
# cannot represent multimodality or long-range correlations in weight space
# the way NUTS's samples can. That shows up here as a modest RMSE/R2 gap
# versus notebook 1's full posterior.
#
# **When to reach for which.** Prefer NUTS/Turing (notebook 1) when the
# network is small enough to sample and the uncertainty quality itself is
# the deliverable (e.g. reserve risking). Prefer LaplaceRedux when you
# already have — or need to retrain often — an ordinary Flux model and want
# calibrated-enough uncertainty bolted on cheaply, or when the network is
# too large for MCMC to be practical at all.
#
# As in notebook 1, this uncertainty is per-prediction (epistemic, from the
# weight posterior) rather than hand-tuned, and the same partial-dependence
# and held-out-well diagnostics apply.
#"""
```